

Options Construction Engine

Locked Implementation Specification

Field	Value
Document type	Technical production specification
Audience	Engineering, research, risk, execution, QA, and internal reviewers
Primary purpose	Convert approved strategy-family and contract-month decisions into deterministic option-leg constructions
Scope	Expiry resolution, strike candidate generation, liquidity validation, pricing and risk validation, candidate ranking, output schema, logging, and acceptance fixtures
Non-scope	Strategy-family decision, contract-month decision, capital allocation, final quantity sizing, broker order placement, stop-loss management, rollover, and exit logic
Normative posture	Implementation-locking. Any change to formulas, thresholds, schema, tie-breaks, rejection rules, or output contract is a controlled specification change.

1. Executive Summary

The Options Construction Engine is a deterministic downstream layer that consumes a published Strategy Engine payload and a validated option chain. It emits either one execution-ready option-leg construction or one structured rejection record. It must not alter the upstream strategy family, contract month, trend view, or strength. Its responsibility is to translate an approved abstract strategy into concrete option legs that pass expiry, strike, liquidity, pricing, risk, and schema checks.

This specification is intended for implementation. It locks boundaries, input contracts, option-chain requirements, selection precedence, formulas, rejection taxonomy, audit logging, order-intent boundaries, and deterministic acceptance tests.

2. Locked Boundaries

Boundary	Required rule
Upstream authority	Strategy Engine output is authoritative for instrument, strategy_family, contract_month selection, and final_strategy_strength.
No override	This engine must not change the strategy family, contract month, or upstream strength.
No execution	This engine may generate order-intent payloads but must not place broker orders.
No final sizing	Quantity and capital allocation belong to the Risk and Portfolio Engine. This engine may compute risk per lot for downstream use.
No stale fallback	If required market data is missing, stale, or invalid, construction must be rejected. Previous option-chain data must not be substituted.
No cost or margin optimization	Brokerage, taxes, slippage modeling, margin optimization, and portfolio exposure controls are outside this layer.

3. Required Inputs

Field	Type	Required rule	Failure code
instrument	string	Non-empty stock derivatives symbol or instrument key.	INPUT_MISSING_INSTRUMENT
asof_time	ISO datetime	Must match the upstream Strategy Engine evaluation timestamp.	INPUT_INVALID_ASOF_TIME

strategy_family	enum	One of BULL_CALL_SPREAD, BEAR_PUT_SPREAD, BULL_PUT_SPREAD, BEAR_CALL_SPREAD, IRON_CONDOR.	INPUT_INVALID_STRATEGY_FAMILY
contract_month_selection	enum	NEAR_MONTH or NEXT_MONTH only.	INPUT_INVALID_CONTRACT_MONTH
final_strategy_strength	integer	0 to 100 inclusive.	INPUT_INVALID_STRENGTH
underlying_spot_price	number	Finite and greater than zero.	INPUT_INVALID_SPOT
lot_size	integer	Positive exchange lot size.	INPUT_INVALID_LOT_SIZE
strike_step	number	Positive exchange strike interval.	INPUT_INVALID_STRIKE_STEP
option_chain	array	Non-empty list of option contracts for the instrument.	OPTION_CHAIN_MISSING

4. Option Chain Contract

Field	Type	Rule
tradingsymbol	string	Non-empty broker tradingsymbol.
instrument_token	integer or string	Required for broker mapping when available.
expiry	date	Valid exchange expiry date.
strike	number	Positive and aligned to strike_step.
option_type	enum	CE or PE.
bid_price	number	Finite and ≥ 0 .
ask_price	number	Finite and ≥ 0 .
last_price	number	Finite and ≥ 0 ; not used for construction economics when bid/ask are available.
open_interest	integer	≥ 0 .
volume	integer	≥ 0 .
delta	number or null	Optional. If present, delta must originate from the same option-chain snapshot source as bid, ask, OI, and volume. Mixed, recomputed, or stale delta values are not permitted. If delta source cannot be verified, delta must be treated as unavailable and distance-based construction must be used.
data_timestamp	ISO datetime	Must satisfy the freshness rule.

5. Freshness and Validation Rules

During live market construction, option-chain data must not be older than 5 minutes. For end-of-day simulation, option-chain data must belong to the same completed trading session. The active freshness policy must be logged with every construction decision.

Check	Rule	Reason code
Option chain freshness	Every candidate leg must use contracts from the same chain snapshot or compatible timestamp batch.	OPTION_CHAIN_STALE
Delta validity	Delta may be used only if delta values are available for all candidate legs, share the same data timestamp as the option chain, and are fresh. Otherwise, distance-based construction must be used and fallback details must be recorded in strike_mode_metadata.	Not emitted from this validation check
Same expiry	All legs in a spread must use the selected expiry.	EXPIRY_MISMATCH
Same underlying	All legs must match the instrument.	UNDERLYING_MISMATCH

Same lot size	All selected legs must share the same lot size.	LOT_SIZE_MISMATCH
Broker tradability	Every selected tradingsymbol must be present in the broker instrument universe.	BROKER_INSTRUMENT_NOT_FOUND

Strike mode selection is governed by Stage 4 — Strike Mode Selection. This section defines only the validation conditions for delta usability and must not emit a reason code independently.

6. Expiry Selection

Expiry selection is derived from the upstream contract_month_selection. Weekly expiries are disabled by default. Monthly expiry means the exchange-recognized monthly stock-option expiry for the selected symbol.

Input month	Selection rule	Failure code
NEAR_MONTH	Choose the nearest valid monthly expiry on or after asof time.	NO_VALID_NEAR_EXPIRY
NEXT_MONTH	Choose the next valid monthly expiry strictly after the near-month monthly expiry.	NO_VALID_NEXT_EXPIRY

7. Strategy to Leg Mapping

Strategy family	Leg roles	Required relationship
BULL_CALL_SPREAD	BUY CE lower strike; SELL CE higher strike	long_strike < short_strike
BEAR_PUT_SPREAD	BUY PE higher strike; SELL PE lower strike	short_strike < long_strike
BULL_PUT_SPREAD	SELL PE higher strike; BUY PE lower strike	long_strike < short_strike
BEAR_CALL_SPREAD	SELL CE lower strike; BUY CE higher strike	short_strike < long_strike
IRON_CONDOR	BUY PE low wing; SELL PE short; SELL CE short; BUY CE high wing	put_long < put_short < spot < call_short < call_long

8. Strike Candidate Generation

Candidate generation must be deterministic. The engine generates the complete candidate universe, evaluates all candidates in deterministic order, applies filters, scores surviving candidates, and chooses the top-ranked construction. Selection must be based on final ranking and not on the first valid candidate. Delta is preferred only when all delta-validity rules pass. Delta and distance must not be mixed within one construction except as logged tie-break metadata.

Candidate Ordering Rule

Candidate evaluation order must be deterministic. Candidates must be sorted by increasing absolute deviation from the target delta when delta mode is used, or by increasing absolute deviation from the target OTM percentage when distance mode is used.

If two candidates have the same target deviation, the candidate with the lower average bid-ask spread percentage must be evaluated first.

If still tied, the candidate with higher combined open interest must be evaluated first.

If still tied, the candidate with higher combined volume must be evaluated first.

If still tied, the lexicographically smaller ordered leg tradingsymbol tuple must be evaluated first.

This ordering rule must be applied before liquidity, pricing, risk, and construction scoring filters.

Candidate Universe Definition

Candidate generation must be deterministic and complete before ordering.

For the selected strategy_family and expiry, the engine must generate the full candidate universe consisting of all strike combinations that satisfy:

1. the strategy-to-leg relationships defined in Section 7,
2. the applicable delta band when DELTA mode is used,
3. the applicable OTM percentage band when DISTANCE mode is used,
4. the width target and acceptable width constraints defined in this section,
5. the strike rounding and candidate expansion rules defined in Section 9.

No candidates outside these bounds may be generated.

Candidate ordering must be applied only after the complete candidate universe has been generated.

Strategy group	Primary target if delta valid	Fallback target if delta unavailable or invalid	Width target
Credit spreads: BULL_PUT_SPREAD, BEAR_CALL_SPREAD	Short leg absolute delta target = 0.20; acceptable band 0.15 to 0.30.	Short leg 5% OTM from spot; acceptable band 3% to 7% OTM.	max(2 strike steps, nearest strike distance to 2% of spot).
Debit spreads: BULL_CALL_SPREAD, BEAR_PUT_SPREAD	Long leg absolute delta target = 0.50; acceptable band 0.45 to 0.60. Short leg absolute delta target = 0.30.	Long leg nearest ATM; short leg 3% away; acceptable band 2% to 5%.	max(2 strike steps, nearest strike distance to 3% of spot).
IRON_CONDOR	Short legs absolute delta target = 0.18; acceptable band 0.12 to 0.25.	Short PE 5% below spot; short CE 5% above spot; acceptable band 3% to 7%.	Wings = 2 strike steps beyond short legs by default.

Acceptable width tolerance is ± 1 strike step from the computed width target unless explicitly configured.

9. Deterministic Strike Rounding Rules

Function	Rule
ATM strike	Nearest listed strike to spot. Tie goes to lower strike for CE debit long leg and higher strike for PE debit long leg.
OTM call target	Round target price upward to the next listed strike greater than spot.
OTM put target	Round target price downward to the next listed strike lower than spot.
Long wing for call credit spread	Choose the first valid listed strike $\geq$ short_call_strike + width_target.
Long wing for put credit spread	Choose the first valid listed strike $\leq$ short_put_strike - width_target.
Candidate expansion	If the target candidate fails, expand one strike step at a time outward, then inward only within the accepted band. Maximum expansion limit is 5 strike steps outward and 3 strike steps inward. If no valid candidate is found within this limit, terminate with NO_VALID_STRIKE_PAIR.

10. Liquidity Filters

Liquidity filters are applied per leg before spread-level pricing. A candidate spread is rejected if any leg fails a hard leg-level liquidity check.

Metric	Default rule	Formula / note
Bid and ask	bid_price > 0 and ask_price > 0	Executable quote must exist.
Open interest	open_interest >= 500	Configurable per symbol tier, but active threshold must be logged.

Volume	volume >= 100	Configurable per symbol tier, but active threshold must be logged.
Bid-ask spread pct	<= 15%	(ask - bid) / mid_price, where mid_price = (bid + ask) / 2.
Quote sanity	ask_price >= bid_price	Reject crossed or invalid quotes.

11. Spread Pricing and Risk Formulas

Pricing must use conservative executable assumptions: sell legs at bid and buy legs at ask. Last traded price must not be used for construction economics.

Spread type	Net premium	Max loss	Max profit	Minimum acceptance
Credit spread	sum(sell_bid) sum(buy_ask)	- width_value - net_credit	net_credit	net_credit >= 0.20 * width_value
Debit spread	sum(buy_ask) sum(sell_bid)	- net_debit	width_value - net_debit	max_profit / max_loss >= 1.0
Iron condor	sell_put_bid + sell_call_bid - buy_put_ask - buy_call_ask	max(put_width, call_width) - net_credit	net_credit	net_credit >= 0.20 * max(put_width, call_width)

width_value is the absolute strike distance between the short leg and protective long leg, expressed per unit before lot-size multiplication. Lot-level risk is max_loss * lot_size. Sizing is not performed here, but the engine must compute lot-level risk metrics for downstream use.

12. Transaction Cost and Margin Boundary

Transaction costs, brokerage, taxes, exchange charges, slippage modeling, margin optimization, and portfolio-level exposure constraints are not included in construction_score. They belong to the Risk and Portfolio Engine. This engine may expose raw premium, width, and lot-level risk values so the downstream risk layer can make those decisions deterministically.

13. Construction Scoring

construction_score is used only to choose among valid constructions. It must not override hard filters. Every sub-score is clipped to [0,1]. Construction scoring must only be applied to candidates that pass all prior validation stages. No candidate failing liquidity, pricing, or risk validation may enter scoring.

Sub-score	Weight	Formula
liquidity_score	35%	Average of leg_liquidity_score across legs. leg_liquidity_score = 0.50*oi_score + 0.30*volume_score + 0.20*quote_validity_score. oi_score = clip(open_interest / 2000). volume_score = clip(volume / 500). quote_validity_score = 1 if bid/ask valid else 0.
strike_fit_score	25%	Delta mode: 1 - abs(abs(delta)-target_delta)/target_tolerance. Distance mode: 1 - abs(actual_otm_pct-target_otm_pct)/allowed tolerance.
reward_risk_score	20%	Credit: clip((net_credit / width_value) / 0.35). Debit: clip(reward_risk_ratio / 2.0).
bid_ask_quality_score	10%	1 - clip(avg bid_ask_spread_pct / 0.15).
expiry_fit_score	10%	1 if expiry exactly matches selected monthly expiry; otherwise 0. No alternate expiry is allowed by default.

```

construction_score = round(100 * clip01(
    0.35*liquidity_score +
    0.25*strike_fit_score +
    0.20*reward_risk_score +
    0.10*bid_ask_quality_score +
    0.10*expiry_fit_score
))

```

14. Candidate Ranking and Tie-Breaks

Order	Tie-break
1	Higher construction_score
2	Higher liquidity_score
3	Lower average bid_ask_spread_pct
4	Closer to target delta or target distance
5	Higher reward_risk_score
6	Lower lot-level max_loss
7	Lexicographically smaller ordered leg tradingsymbol tuple for deterministic final tie-break

15. Output Contract

The implementation must emit a schema-conformant object. Rejection records are construction outputs and must include errors. Successful constructions must include errors as an empty list. Hard failure means no constructed trade; a structured REJECTED construction record is still emitted.

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Options Construction Engine Output",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "spec_identifier", "instrument", "asof_time", "strategy_family",
    "contract_month_selection", "construction_status", "execution_ready",
    "reason_codes", "errors"
  ],
  "properties": {
    "spec_identifier": {"const": "options-construction-engine-locked-specification"},
    "instrument": {"type": "string", "minLength": 1},
    "asof_time": {"type": "string", "format": "date-time"},
    "strategy_family": {"enum": ["BULL_CALL_SPREAD", "BEAR_PUT_SPREAD", "BULL_PUT_SPREAD", "BEAR_CALL_SPREAD",
"IRON_CONDOR"]},
    "contract_month_selection": {"enum": ["NEAR_MONTH", "NEXT_MONTH"]},
    "expiry": {"type": ["string", "null"], "format": "date"},
    "legs": {
      "type": "array",
      "items": {
        "type": "object",
        "additionalProperties": false,
        "required": ["role", "side", "option_type", "strike", "expiry", "trading_symbol", "instrument_token",
"bid_price", "ask_price"],
        "properties": {
          "role": {"enum": ["LONG_LEG", "SHORT_LEG", "PUT_LONG_WING", "PUT_SHORT", "CALL_SHORT", "CALL_LONG_WING"]},
          "side": {"enum": ["BUY", "SELL"]},
          "option_type": {"enum": ["CE", "PE"]},
          "strike": {"type": "number", "exclusiveMinimum": 0},
          "expiry": {"type": "string", "format": "date"},
          "trading_symbol": {"type": "string", "minLength": 1},

```

```

    "instrument_token": {"type": ["integer", "string"]},
    "bid_price": {"type": "number", "minimum": 0},
    "ask_price": {"type": "number", "minimum": 0}
  }
},
"net_premium": {"type": ["number", "null"]},
"max_loss_per_lot": {"type": ["number", "null"]},
"max_profit_per_lot": {"type": ["number", "null"]},
"construction_score": {"type": ["integer", "null"], "minimum": 0, "maximum": 100},
"execution_ready": {"type": "boolean"},
"construction_status": {"enum": ["CONSTRUCTED", "REJECTED"]},
"reason_codes": {"type": "array", "items": {"type": "string", "uniqueItems": true},
"errors": {"type": "array", "items": {"type": "object", "required": ["code", "message"]}}
}
}

```

Audit-only metadata, including strike_mode_metadata, delta source metadata, candidate identity, candidate lists, rejected-candidate diagnostics, and idempotency evidence, must be stored in the audit record defined in Section 18 and must not be added to the public construction output unless the output schema is formally revised.

16. Reason Codes

Category	Codes
Input	INPUT_MISSING_INSTRUMENT, INPUT_INVALID_ASOF_TIME, INPUT_INVALID_STRATEGY_FAMILY, INPUT_INVALID_CONTRACT_MONTH, INPUT_INVALID_STRENGTH, INPUT_INVALID_SPOT, INPUT_INVALID_LOT_SIZE, INPUT_INVALID_STRIKE_STEP
Option chain	OPTION_CHAIN_MISSING, OPTION_CHAIN_STALE, BROKER_INSTRUMENT_NOT_FOUND, UNDERLYING_MISMATCH, EXPIRY_MISMATCH, LOT_SIZE_MISMATCH
Expiry	EXPIRY_SELECTED_NEAR_MONTH, EXPIRY_SELECTED_NEXT_MONTH, NO_VALID_NEAR_EXPIRY, NO_VALID_NEXT_EXPIRY
Selection	DELTA_BASED_STRIKE_SELECTION, DISTANCE_BASED_STRIKE_SELECTION, NO_VALID_STRIKE_PAIR
Liquidity	LIQUIDITY_CHECK_PASSED, LIQUIDITY_CHECK_FAILED, BID_ASK_SPREAD_TOO_WIDE, OPEN_INTEREST_TOO_LOW, VOLUME_TOO_LOW
Pricing/risk	PREMIUM_CHECK_PASSED, CREDIT_TOO_LOW, POOR_REWARD_RISK, INVALID_MAX_LOSS, INVALID_NET_PREMIUM
Runtime	OUTPUT_SCHEMA_VALIDATION_FAILED, DECISION_LOG_WRITE_FAILED, CONSTRUCTION_SUCCESS

17. Deterministic Reason Code Mapping

Reason codes must be emitted deterministically at the exact stage where a decision or failure occurs. At each stage, exactly one primary reason code must be emitted to represent the outcome of that stage. Secondary diagnostic codes may be recorded in logs but must not override the primary reason code.

Stage 1 — Input Validation

Condition	Primary Reason Code
instrument missing or empty	INPUT_MISSING_INSTRUMENT

asof_time missing, invalid, or mismatched with upstream evaluation timestamp	INPUT_INVALID_ASOF_TIME
strategy_family missing or outside supported enum	INPUT_INVALID_STRATEGY_FAMILY
contract_month_selection missing or invalid	INPUT_INVALID_CONTRACT_MONTH
final_strategy_strength outside [0, 100]	INPUT_INVALID_STRENGTH
underlying_spot_price non-finite or ≤ 0	INPUT_INVALID_SPOT
lot_size ≤ 0 or invalid	INPUT_INVALID_LOT_SIZE
strike_step ≤ 0 or invalid	INPUT_INVALID_STRIKE_STEP
option_chain missing or empty	OPTION_CHAIN_MISSING

Stage 2 — Option Chain Validation

Condition	Primary Reason Code
option-chain data violates freshness rule	OPTION_CHAIN_STALE
option-chain contracts derived from inconsistent timestamps	OPTION_CHAIN_STALE
contract underlying does not match input instrument	UNDERLYING_MISMATCH
legs do not share the selected expiry	EXPIRY_MISMATCH
legs do not share the same lot size	LOT_SIZE_MISMATCH
trading symbol not present in broker instrument universe	BROKER_INSTRUMENT_NOT_FOUND

Stage 3 — Expiry Selection

Condition	Primary Reason Code
no valid near-month expiry available	NO_VALID_NEAR_EXPIRY
no valid next-month expiry available	NO_VALID_NEXT_EXPIRY
near-month expiry successfully selected	EXPIRY_SELECTED_NEAR_MONTH
next-month expiry successfully selected	EXPIRY_SELECTED_NEXT_MONTH

Stage 4 — Strike Mode Selection

Condition	Primary Reason Code
all required delta values available, valid, and timestamp-aligned	DELTA_BASED_STRIKE_SELECTION
distance-based construction applied	DISTANCE_BASED_STRIKE_SELECTION

Stage 5 — Candidate Generation

Condition	Primary Reason Code
no valid strike pairs generated within accepted bounds	NO_VALID_STRIKE_PAIR

This stage represents generation failure. NO_VALID_STRIKE_PAIR at this stage means no valid candidate strike combinations could be produced within the defined target bands before any filtering.

Stage 6 — Liquidity Filtering

Priority	Condition	Primary Reason Code
1	$\text{bid_price} \leq 0$ or $\text{ask_price} \leq 0$	LIQUIDITY_CHECK_FAILED
2	$\text{ask_price} < \text{bid_price}$	LIQUIDITY_CHECK_FAILED
3	bid-ask spread exceeds configured threshold	BID_ASK_SPREAD_TOO_WIDE
4	open_interest below active threshold	OPEN_INTEREST_TOO_LOW
5	volume below active threshold	VOLUME_TOO_LOW
6	all legs pass liquidity validation	LIQUIDITY_CHECK_PASSED

If multiple liquidity conditions fail for the same leg or candidate, the primary reason code must be selected based on the lowest numerical priority value. All other failed conditions must be recorded in audit logs but must not override the primary reason code.

Stage 7 — Pricing Validation

Condition	Primary Reason Code
-----------	---------------------

credit spread net_credit $\leq$ 0	INVALID_NET_PREMIUM
credit spread net_credit below minimum threshold	CREDIT_TOO_LOW
debit spread net_debit $\leq$ 0	INVALID_NET_PREMIUM
pricing validation successful	PREMIUM_CHECK_PASSED

Stage 8 — Risk Validation

Condition	Primary Reason Code
max_loss cannot be computed or is invalid	INVALID_MAX_LOSS
debit spread reward-to-risk below threshold	POOR_REWARD_RISK

Stage 9 — Candidate Exhaustion

Condition	Primary Reason Code
all generated candidates rejected after validation	NO_VALID_STRIKE_PAIR

This stage represents post-filter exhaustion. NO_VALID_STRIKE_PAIR at this stage indicates that candidate strike pairs were generated, but all candidates were rejected during liquidity, pricing, or risk validation.

Stage 10 — Output and Runtime Validation

Condition	Primary Reason Code
output schema validation fails	OUTPUT_SCHEMA_VALIDATION_FAILED
decision logging fails	DECISION_LOG_WRITE_FAILED

Stage 11 — Successful Construction

Condition	Primary Reason Code
construction completes successfully with valid output	CONSTRUCTION_SUCCESS

NO_VALID_STRIKE_PAIR is used in both Stage 5 (generation failure) and Stage 9 (post-filter exhaustion). Stage context determines the failure origin.

Enforcement Rule

Exactly one primary reason code must be emitted per stage.

Failure codes must terminate further processing unless explicitly defined as non-terminal.

Informational codes must not override failure codes.

Reason code emission must be deterministic for identical inputs.

When multiple conditions fail within the same stage, the primary reason code must be selected using the explicit priority order defined for that stage.

If a stage does not define a separate priority table, the primary reason code must be selected by the order of conditions listed in that stage's mapping table, from top to bottom.

Only the highest-priority failure code must be emitted as the primary reason code for that stage. Other failed conditions may be recorded in audit diagnostics, but must not override the primary reason code.

Success reason codes must be emitted only if no failure condition applies within that stage.

18. Logging and Audit Requirements

Each evaluation must create an audit record, even if rejected. The audit record must be machine-readable and sufficient to reproduce the decision.

Audit item	Requirement
Input snapshot	Store upstream payload fields, spot, lot size, strike step, option-chain snapshot identifier, and freshness metadata.
Candidate list	Store generated candidates before and after each filter stage.
Strike mode metadata	Store selected strike mode as DELTA or DISTANCE. If DISTANCE is used due to unavailable, stale, incomplete, or invalid delta values, log delta_available, delta_valid, and fallback_reason as one of MISSING, STALE, INCOMPLETE, or INVALID.
Rejected candidates	Store candidate id and exact rejection codes.
Final selection	Store selected legs, economics, scoring subcomponents, final score, and tie-break values.
Implementation identity	Store specification identifier and implementation build hash.
Failure isolation	Failure for one instrument must not alter another construction result.
Idempotency	For identical inputs (instrument, asof_time, and option-chain snapshot), the engine must produce identical outputs including construction_score, selected legs, and reason_codes. Repeated evaluations must not produce different candidates or rankings unless input data changes.
Delta source metadata	Store delta_source, delta_timestamp, and delta_source_verified whenever DELTA mode is used.
Candidate identity	Candidate identity must be derived from strategy_family, expiry, ordered leg roles, ordered leg sides, ordered leg strikes, ordered leg option types, and ordered leg tradingsymbols.

19. Broker Order-Intent Contract

This layer emits order-intent only. It must not set final quantity, final executable limit price, or place orders. Those responsibilities belong to downstream risk and execution layers.

Field	Rule
exchange	NFO for NSE derivatives unless configured otherwise.
tradingsymbol	Must match selected option-chain contract.
transaction_type	BUY or SELL from leg side.

quantity	Null or one lot placeholder unless Risk Engine supplies quantity. Final quantity must not be decided here.
product	Default NRML; configurable but logged.
order_type	Default LIMIT for live execution intent. Final executable price must be set by the execution layer.
validity	DAY unless configured otherwise.
variety	regular or basket-equivalent metadata; final value belongs to execution layer.
execution_sequence	Protective buy legs should be prioritized before naked sell exposure when converted to actual orders by the execution layer.

20. Pseudocode

```

function construct_options(strategy_payload, option_chain, market_context):
    validated = validate_inputs(strategy_payload, option_chain, market_context)
    if not validated.ok:
        return rejected_output(validated.errors)

    expiry = select_monthly_expiry(strategy_payload.contract_month_selection, option_chain)
    if expiry is null:
        return rejected_output(NO_VALID_EXPIRY)

    mode = choose_strike_mode(option_chain)
    # DELTA only if all required deltas are fresh and timestamp-aligned; otherwise DISTANCE.

    candidates = generate_candidates(strategy_payload.strategy_family, expiry, mode, market_context)

    candidates = sort_by_candidate_ordering_rule(candidates)

    candidates = apply_leg_liquidity_filters(candidates)
    candidates = apply_spread_pricing_filters(candidates)
    candidates = apply_risk_filters(candidates)

    if candidates is empty:
        return rejected_output(NO_VALID_STRIKE_PAIR)

    scored = compute_construction_scores(candidates)
    selected = sort_by_locked_tiebreaks(scored)[0]
    output = build_success_output(selected)
    validate_output_schema(output)
    log_decision(output, candidates)
    return output

```

21. Acceptance Test Fixtures

Test id	Scenario	Expected result
TC-001	BEAR_CALL_SPREAD, NEXT_MONTH, spot 150, valid chain, delta available, short CE delta closest to 0.20, long CE two strike steps above.	CONSTRUCTED with SELL CE lower strike and BUY CE higher strike; expiry next monthly.
TC-002	Same as TC-001 but selected short leg bid = 0.	Reject that candidate; evaluate next deterministic candidate.
TC-003	All candidate CE spreads fail bid-ask spread <= 15%.	REJECTED with LIQUIDITY_CHECK_FAILED and BID_ASK_SPREAD_TOO_WIDE.
TC-004	BULL_PUT_SPREAD with Greeks unavailable.	Use DISTANCE_BASED_STRIKE_SELECTION and log fallback_reason under strike_mode_metadata.
TC-005	Credit spread net_credit = 10% of width.	REJECTED with CREDIT_TOO_LOW.
TC-006	Debit spread reward/risk = 0.8.	REJECTED with POOR_REWARD_RISK.

TC-007	Identical candidates except bid-ask quality.	Select lower average bid-ask spread after construction_score tie.
TC-008	No next monthly expiry for NEXT_MONTH input.	REJECTED with NO_VALID_NEXT_EXPIRY.

22. Reference Input and Expected Output Fixture

This section provides a complete, deterministic reference fixture including a valid input payload and the expected output. This fixture is intended to validate end-to-end correctness of implementation, including expiry selection, strike construction, liquidity validation, pricing, scoring, and output schema compliance.

22.1 Reference Input

```
{
  "instrument": "ABC",
  "asof_time": "2026-04-15T10:30:00Z",
  "strategy_family": "BEAR_CALL_SPREAD",
  "contract_month_selection": "NEXT_MONTH",
  "final_strategy_strength": 72,
  "underlying_spot_price": 150.0,
  "lot_size": 100,
  "strike_step": 5,
  "option_chain": [
    {
      "trading_symbol": "ABC24MAY155CE",
      "instrument_token": 1002,
      "expiry": "2026-05-28",
      "strike": 155,
      "option_type": "CE",
      "bid_price": 3.0,
      "ask_price": 3.5,
      "last_price": 3.2,
      "open_interest": 2200,
      "volume": 700,
```

```

    "delta": 0.20,

    "data_timestamp": "2026-04-15T10:29:00Z"
  },

  {

    "trading_symbol": "ABC24MAY160CE",

    "instrument_token": 1003,

    "expiry": "2026-05-28",

    "strike": 160,

    "option_type": "CE",

    "bid_price": 1.5,

    "ask_price": 2.0,

    "last_price": 1.7,

    "open_interest": 1800,

    "volume": 600,

    "delta": 0.10,

    "data_timestamp": "2026-04-15T10:29:00Z"
  }
]
}

```

22.2 Expected Output

```

{

  "spec_identifier": "options-construction-engine-locked-specification",

  "instrument": "ABC",

  "asof_time": "2026-04-15T10:30:00Z",

  "strategy_family": "BEAR_CALL_SPREAD",

  "contract_month_selection": "NEXT_MONTH",

  "expiry": "2026-05-28",

  "legs": [

    {

```

```

"role": "SHORT_LEG",

"side": "SELL",

"option_type": "CE",

"strike": 155,

"expiry": "2026-05-28",

"trading_symbol": "ABC24MAY155CE",

"instrument_token": 1002,

"bid_price": 3.0,

"ask_price": 3.5

},

{

"role": "LONG_LEG",

"side": "BUY",

"option_type": "CE",

"strike": 160,

"expiry": "2026-05-28",

"trading_symbol": "ABC24MAY160CE",

"instrument_token": 1003,

"bid_price": 1.5,

"ask_price": 2.0

}

],

"net_premium": 1.0,

"max_loss_per_lot": 400.0,

"max_profit_per_lot": 100.0,

"construction_score": 81,

"execution_ready": true,

"construction_status": "CONSTRUCTED",

"reason_codes": [

"EXPIRY_SELECTED_NEXT_MONTH",

```

```

"DELTA_BASED_STRIKE_SELECTION",

"LIQUIDITY_CHECK_PASSED",

"PREMIUM_CHECK_PASSED",

"CONSTRUCTION_SUCCESS"

],

"errors": []

}

```

22.3 Notes

- Pricing uses conservative assumptions: sell at bid and buy at ask.
- $\text{net_premium} = 3.0 - 2.0 = 1.0$ per unit.
- $\text{width_value} = 160 - 155 = 5$.
- $\text{max_loss_per_lot} = (\text{width_value} - \text{net_premium}) \times \text{lot_size} = (5 - 1.0) \times 100 = 400$.
- $\text{max_profit_per_lot} = \text{net_premium} \times \text{lot_size} = 1.0 \times 100 = 100$.
- $\text{construction_score} = 81$, computed as per Section 13 using the fixture values.
- reason_codes reflect one primary code per applicable stage.
- This fixture must be reproducible given identical input and option-chain snapshot.

23. Known Limitations and Downstream Dependencies

This specification is implementation-ready for deterministic option-leg construction. It intentionally excludes portfolio-level sizing, margin optimization, tax and transaction-cost modeling, implied-volatility regime selection, stop-loss management, rollover, and live order placement. Those responsibilities must be handled by the Risk and Portfolio Engine, Execution Engine, and Position Monitoring Engine.